

CSC 1204: Data Structures and Algorithms

J. Kizito

Makerere University

e-mail: john.kizito@mak.ac.ug
www: <https://www.socnet.co.ug/~jona>
materials: <https://www.socnet.co.ug/~jona/materials/CSC1204>
e-learning environment: <http://muele.mak.ac.ug>
office: block A, level 3, department of computer science
alt. office: institute of open, distance, and eLearning

Searching Algorithms



Overview

- 1 Searching Algorithms
 - Linear Search
 - Binary Search
 - Interpolation Search
 - Hashing
- 2 Algorithm Comparison
- 3 Algorithm Implementation



Searching Algorithms

- A search algorithm is an algorithm for finding an item with specified properties among a collection of items
- A search typically answers either *True* or *False* as to whether the item is present
- On occasion it may be modified to return where the item is found

Popular Algorithms

- 1 Linear (Sequential) search
- 2 Binary (Half-interval) search
- 3 Interpolation search
- 4 Hashing



Searching Algorithms

Linear Search



- Search through the whole list from one end to the other
- Starting at the first item in the list, we simply move from item to item, following the underlying sequential ordering until we either find what we are looking for or run out of items
- If we run out of items, we have discovered that the item we were searching for was not present
- Best case: 1; Worst case: n ; Average case: $n/2$



Linear Search

Algorithms (1)

Unordered List

```
def sequentialSearch(alist, item):  
    pos = 0  
    found = False  
  
    while pos < len(alist) and not found:  
        if alist[pos] == item:  
            found = True  
        else:  
            pos = pos+1  
  
    return found
```



Linear Search

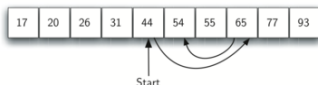
Algorithms (2)

Ordered List

```
def orderedSequentialSearch(alist, item):  
    pos = 0  
    found = False  
    stop = False  
    while pos < len(alist) and not found and not stop:  
        if alist[pos] == item:  
            found = True  
        else:  
            if alist[pos] > item:  
                stop = True  
            else:  
                pos = pos+1  
  
    return found
```

Searching Algorithms

Binary Search



- Split the list into two roughly equal sub-lists and search one of the halves
- Makes clever comparisons by taking advantage of the ordered (sorted) list
 - 1 We start by examining the middle item. If that item is the one we are searching for, then we are done
 - 2 Otherwise, we eliminate half of the remaining items, i.e., If the item we are searching for is, say, greater than the middle item, we eliminate the lower half and the middle item. The search item should belong to the upper half
 - 3 We then repeat the process with the selected half

Complexity – Worst Case

- 1st comparison leaves about $n/2$ items; 2nd $\rightarrow n/4$; 3rd $\rightarrow n/8$; ...; m^{th} (last) $\rightarrow 1$
- At the m^{th} comparison, we have $\frac{n}{2^m} = 1$. Solving for m , gives $O(\log n)$

Binary Search

Algorithms (1)

Ordered List of Integers

```
def binarySearch(alist, item):  
    first = 0  
    last = len(alist)-1  
    found = False  
  
    while first<=last and not found:  
        midpoint = (first + last)/2  
        if alist[midpoint] == item:  
            found = True  
        else:  
            if item < alist[midpoint]:  
                last = midpoint-1  
            else:  
                first = midpoint+1  
  
    return found
```


Binary Search

Algorithms (2)

Recursive Version

```
def binarySearch(alist, item):  
    if len(alist) == 0:  
        return False  
    else:  
        midpoint = len(alist)/2  
        if alist[midpoint]==item:  
            return True  
        else:  
            if item<alist[midpoint]:  
                return binarySearch(alist[:midpoint],item)  
            else:  
                return binarySearch(alist[midpoint+1:],item)
```

Searching Algorithms

Interpolation Search

- An improvement to Binary search, sometimes referred to as *Extrapolation* search
- ① Given n sorted items with range $r - l$, the average interval between each 2 items is $\frac{r-l}{n-1}$
- ② The rough position of a number x in the list is $\frac{x-l}{(r-l)/(n-1)}$
- ③ This is the position of splitting and by comparing, we determine which half to deal with
- Best case: 1; Worst case: $\log(\log(n))$



Searching Algorithms

Hashing

0	1	2	3	4	5	6	7	8	9	10
None	None	None	None	None	None	None	None	None	None	None

- A concept that attempts to build a data structure that can be searched in $O(1)$ time
- If every item is where it should be, then the search can use a single comparison to discover the presence of an item. However, this is typically not the case
- This is achieved using a *hash table* and a *hash function*
- Each position of the hash table, often called a slot, can hold an item and is named by an integer value starting at 0
- The mapping between an item and the slot where that item belongs in the hash table is called the *hash function*
- The hash function will take any item in the collection and return an integer in the range of slot names, between 0 and $m - 1$



Hashing

Sample Hash Functions (1)

The Remainder Method

- Takes an item and divides it by the table size (m), returning the remainder as its hash value [$h(item) = item \% m$]
- Now when we want to search for an item, we simply use the hash function to compute the slot name (number) for the item and then check the hash table to see if it is present
- Given a set of integer items: 54, 26, 93, 17, 77, and 31

0	1	2	3	4	5	6	7	8	9	10
77	None	None	None	26	93	17	None	None	31	54

- Note that 6 of the 11 slots are now occupied. This is referred to as the *load factor*, and is commonly denoted by $\lambda = \frac{\text{number of items}(n)}{\text{table size}(m)} = \frac{6}{11}$
- Assuming we have a new value of 44? This is referred to as a *collision*
- A hash function that maps each item into a unique slot is referred to as a *perfect* hash function

Hashing

Sample Hash Functions (2)

The Folding Method

- Begins by dividing the item into equal-size pieces (the last piece may not be of equal size)
- These pieces are then added together to give the resulting hash value
- Examples
 - 1 $93 \Rightarrow 9 + 3 = 12 \Rightarrow 12 \% 11 = 1$
 - 2 $43658 \Rightarrow 43 + 65 + 8 = 116 \Rightarrow 116 \% 11 = 6$

The Mid-Square Method

- First square the item, and then extract some portion of the resulting digits
- Examples
 - 1 $44 \Rightarrow 44^2 = 1,936 \Rightarrow$ Extract the middle two digits, $93 \Rightarrow 93 \% 11 = 5$
 - 2 $77 \Rightarrow 77^2 = 5,929 \Rightarrow 92 \% 11 = 4$
 - 3 $54 \Rightarrow 54^2 = 2,916 \Rightarrow 91 \% 11 = 3$

Hashing

Comparison of Remainder, Mid-Square, and Folding Methods

Item	Remainder	Mid-Square	Folding
54	10	3	9
26	4	7	8
93	5	9	1
17	6	8	8
77	0	4	3
31	9	6	4

- We can also create hash functions for character-based items such as strings
- E.g., the string “cat” can be thought of as a sequence of ascii values: 99 97 116
- We can then take these three values, add (or concatenate) them and use any of the methods above to get a hash value



Hashing

Comparison of Remainder, Mid-Square, and Folding Methods

Item	Remainder	Mid-Square	Folding
54	10	3	9
26	4	7	8
93	5	9	1
17	6	8	8
77	0	4	3
31	9	6	4

- We can also create hash functions for character-based items such as strings
- E.g., the string “cat” can be thought of as a sequence of ascii values: 99 97 116
- We can then take these three values, add (or concatenate) them and use any of the methods above to get a hash value



Hashing

Collision Resolution

- Recall: *If the hash function is perfect, collisions will never occur*
- When two items hash to the same slot, we must have a systematic method for placing the second item in the hash table

Collision Resolution Methods

- 1 Open Addressing (Linear Probing)
- 2 Plus 3
- 3 Quadratic Probing
- 4 Chaining



Collision Resolution

Linear Probing

- Start at the original hash value position and sequentially try to find the next open slot in the hash table. Note that we may need to go back to the first slot (circularly) to cover the entire table
- Original set of items: 54, 26, 93, 17, 77, and 31:

0	1	2	3	4	5	6	7	8	9	10
77	None	None	None	26	93	17	None	None	31	54

- Extended set: 54, 26, 93, 17, 77, 31, 44, 55, 20

0	1	2	3	4	5	6	7	8	9	10
77	44	55	20	26	93	17	None	None	31	54

- Once a hash table is built using a given method, it is essential that we utilize the same methods to search for items
- E.g., Assuming we are looking for 20? The hash value is 9, which is holding 31. We cannot simply return False since we know that there could have been collisions



Collision Resolution

Plus 3

- A disadvantage to linear probing is the tendency for *clustering*
- E.g., A Cluster of Items for Slot 0:

0	1	2	3	4	5	6	7	8	9	10
77	44	55	20	26	93	17	None	None	31	54

- One way to deal with clustering is to extend the technique so that instead of looking sequentially for the next open slot, we skip some slots, thereby more evenly distributing the items that have caused collisions
- “Plus 3” probe means that once a collision occurs, we will look at every third slot until we find one that is empty

0	1	2	3	4	5	6	7	8	9	10
77	55	None	44	26	93	17	20	None	31	54

- The process of looking for another slot after a collision is called *rehashing*
- To ensure that part of the table is never unused, it is often suggested that the table size be a prime number



Collision Resolution

Quadratic Probing

- Another variation of the linear probing idea is called *quadratic probing*
- Instead of using a constant “skip” value, we use a *rehash* function that increments the hash value by 1, 3, 5, 7, 9, and so on
- i.e., if the first hash value is h , the successive values are $h + 1$, $h + 4$, $h + 9$, $h + 16$, and so on
- In other words, quadratic probing uses a skip consisting of successive perfect squares

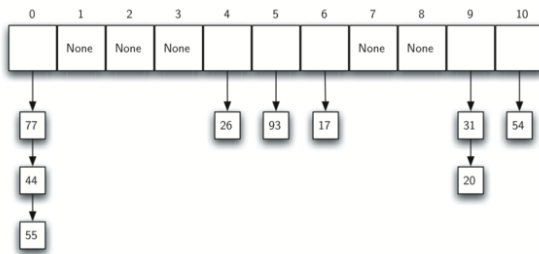
0	1	2	3	4	5	6	7	8	9	10
77	44	20	55	26	93	17	None	None	31	54



Collision Resolution

Chaining

- An alternative method is to allow each slot to hold a reference to a collection (or chain) of items
- This allows many items to exist at the same location in the hash table
- The search is perhaps more efficient since on the average there are likely to be many fewer items in each slot



Algorithm Comparison

Name	Best	Average	Worst
Hashing (linear probing)	1	$\frac{1}{2}(1 + \frac{1}{1-\lambda})$	$\frac{1}{2}(1 + (\frac{1}{1-\lambda})^2)$
Hashing (chaining)	1	$1 + \frac{\lambda}{2}$	λ
Interpolation Search	1	$\log \log n$	
Binary Search	1		$\log n$
Linear Search	1	$\frac{n}{2}$	n
Depth First Search (DFS)	$ V $		$ E + V $
Breadth First Search (BFS)	$ V $		$ E + V $

Challenge for the Bored

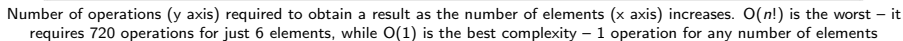
- 1 Implement all of the algorithms provided for Linear and Binary search in C
- 2 Write a program that Implements the Interpolation search algorithm
- 3 Implement the chaining method of collision resolution using linked lists
- 4 See last slide

Algorithm Comparison

Name	Best	Average	Worst
Hashing (linear probing)	1	$\frac{1}{2}(1 + \frac{1}{1-\lambda})$	$\frac{1}{2}(1 + (\frac{1}{1-\lambda})^2)$
Hashing (chaining)	1	$1 + \frac{\lambda}{2}$	λ
Interpolation Search	1	$\log \log n$	
Binary Search	1		$\log n$
Linear Search	1	$\frac{n}{2}$	n
Depth First Search (DFS)	$ V $		$ E + V $
Breadth First Search (BFS)	$ V $		$ E + V $

Challenge for the Bored

- 1 Implement all of the algorithms provided for Linear and Binary search in C
- 2 Write a program that Implements the Interpolation search algorithm
- 3 Implement the chaining method of collision resolution using linked lists
- 4 See last slide



Breadth-First and Depth-First Search

Graph Traversal Example

Graph Traversal

See powerpoint with details...

Breadth-First Search

- Visits all the nodes at one level of the graph before proceeding to the next level
- Returns the path containing the least number of nodes (the shallowest path)

Depth-First Search

- Performs the pre-order traversal of the graph and returns the leftmost path
- This path could luckily be the shallowest or the deepest if we are unlucky